BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 04

> **Objective & Lecture Mapping:** In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment[cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

> Informed search algorithms require a heuristic function, h(n), which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class[cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula h(n) = |x_1 - x_2| + |y_1 - y_2| and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

> A* evaluates nodes by combining the path cost so far, g(n), and the estimated cost to the goal, h(n). The evaluation function is f(n) = g(n) + h(n).

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses g(n), your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, g(n) = 0. Calculate h(n) using your chosen heuristic method, and set f(n) = g(n) + h(n).
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate g_{new} = g_{current} + 1, h_{new} using your heuristic, and f_{new} = g_{new} + h_{new}. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

> Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

UCS uses just the costs to the next neighboring nodes and chooses the minimum cost path to travel based on the current built up cost as well. A star search considers the current built up cost along with the cheapest cost ahead just like UCS but additionally , it considers a heuristic value (calculated specifically for each node and not accumulated) that it adds with the built up cost to decide what path to choose.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

An admissible heuristic is one that never overestimates the true cost to reach the goal. The Manhattan distance works here as it calculates the heuristic using the sum of the absolute coordinate differences of x and y which will always be equal to or less than (if there are walls in the way) the true cost. Since there is no diagonal movement and the cost of each move up, down, left and right is always one, the minimum possible shortest path between two points will exactly be the difference between the x values and the difference between the y values which is directly what the heuristic calculates so it can never calculate something that overestimates the true cost as its actual calculation gives the minimum true cost possible by definition. The true cost will always be exactly the value of the heuristic or higher if there are walls in the middle.

If the heuristic was not admissible , then the search may consider an optimal path as not optimal because it's cost might be too inflated, so another path may be prioritized over it even if it's cost is lower than the selected path. A path will be selected anyhow but it will be impossible to guarantee that it is always the shortest or least costing path.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No it would stop being admissible if 8 way movement was allowed. The reason is Manhattan distance is calculated based on the assumption that only one of x or y can be changed in one move meaning the sum of the differences of each is used to calculate it. For example moving from (0,0) to (1,1) would result in a 2 from Manhattan distance .
However with diagonal movement , it is possible to move in both x and y at once meaning it would only cost *sqrt2 whereas Manhattan distance would need two steps. Here Euclidean distance can be used as an admissible heuristic as it calculates the straight line distance between two points which is always the minimum true cost when diagonal moving is enabled so it can never be overestimated.
It is important to consider however, that for Euclidean distance to be admissible as a heuristic it would need to calculate the true distance not a label such as one , so if 1 is what a direct vertical or horizontal move costs a diagonal move has to be the mathematical diagonal of sqrt2

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Minimum cost spanning tree. First connect each food position to all its neighbors. Then calculate either the Manhattan or the Euclidean distance for each pair of food nodes (ie. each edge). Then label each edge with its corresponding weight and find the set of edges that connect every food position to ever other food position while costing the least and not looping (ie. the MCST) This tree can then be used as the heuristic.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation



FACULTY OF COMPUTING